LIFEOS

Building an Offline Personal Productivity System

A portfolio case study on combining Python desktop UI, SQLite, timers, analytics and reporting into one focused application.

Python	CustomTkinter	SQLite	Matplotlib	Seaborn	Git

Tushar Thakur

MCA Data Science · Chandigarh University

Project Type Python Mini Project	**Interface** Desktop Application	**Database** SQLite	**Launch** Windows EXE
Modules 11 Core Modules	**Tracking** Focus / Pomodoro / Stopwatch	**Reports** Daily / Weekly / Monthly	**Theme** Light / Dark / System

PROJECT STORY

Why I Built LifeOS

I wanted to build a productivity project that felt like one connected workspace instead of a collection of unrelated screens. LifeOS brings tasks, planning, notes, focused work, time tracking, analytics and reports into a single offline desktop application.

The project became an exercise in state management as much as interface design. Timers need to survive navigation, completed work must update history and reports, and the same local data should power the dashboard, analytics and productivity score.

CONTEXT

The Problem

Fragmented productivity workflow

Tasks, notes, timers and daily plans often live in separate tools, making it harder to see progress or maintain one reliable activity history.

What LifeOS does

Brings planning, focused work, time tracking, notes, analytics and reporting into one offline desktop workflow backed by local data.

KEY CAPABILITIES

What the Application Can Do

Tasks & planner

Create, schedule, prioritize, categorize and complete tasks while planning dated activities on an interactive monthly calendar.

Notes & focus

Capture searchable categorized notes, open full-note popups, and run protected Focus sessions linked to pending tasks.

Pomodoro & stopwatch

Use structured Pomodoro intervals or open-ended stopwatch sessions with pause, resume, laps and saved history.

Dashboard & analytics

See today's work, productivity score, weekly trends, focus time and task-completion charts from real local activity.

History & reports

Filter activity history and generate Daily, Weekly and Monthly reports with completed-task and category summaries.

Settings & local data

Switch themes, configure export folders, back up or restore SQLite data, and keep the application fully offline.

ARCHITECTURE

How LifeOS Is Structured

LifeOS follows a compact desktop architecture: CustomTkinter provides the interface, Python modules coordinate application behavior, SQLite stores productivity data, and analytics/reporting modules turn that data into useful feedback.

User UI → **Page Modules** → **Database Layer** → **SQLite** → **Updated View**

Layer	Main Technology	Responsibility
Interface	CustomTkinter / Tkinter	Responsive desktop pages, forms, cards, dialogs, navigation and theme-aware controls.
Application	Python modules	Timers, validation, page state, productivity rules, workflows and navigation behavior.
Data & insight	SQLite + JSON + Matplotlib	Persistent activity, settings, query helpers, reports and visual analytics.

EXAMPLE WORKFLOW

Data Flow: Completing a Focus Session

Choose task → **Start focus** → **Track timer** → **Stop / finish** → **Save session** → **Refresh metrics** → **Show insights**

A Focus session can be linked to a pending task or run as General focus. Timer state is preserved while pages are cached, and leaving an active session requires confirmation. When the session stops or completes, elapsed time is stored and immediately becomes available to history, analytics and productivity metrics.

CORE RELATIONSHIPS

Local Data Model

Entity	Connection	Usage
Task	Completion state	Dashboard, history and reports
Focus session	Optional task / General	Focus stats and productivity score
Planner activity	Scheduled date	Calendar, dashboard and productivity score
Note	Category	Searchable personal knowledge

LOCAL DATA ENGINEERING

The Data Work Behind the UI

LifeOS uses a deliberately simple local data model. SQLite stores persistent activity, JSON stores application preferences, and small query/helper functions keep each screen focused on presentation rather than raw database handling.

Object	Examples	Purpose
SQLite tables	tasks, notes, planner activities, focus / timer sessions	Persist productivity activity between application launches.
Database helpers	CRUD, filters, statistics and report queries	Keep SQL access reusable and separated from screen layout code.
Settings JSON	appearance mode, export folder, app version	Persist lightweight preferences without requiring another database table.
Analytics helpers	productivity metrics and 7-day summaries	Convert stored activity into scores, charts, dashboard values and reports.

RELIABILITY

State & Data Safety

CACHED PAGES	FOCUS LOCK	DATABASE BACKUP	SAFE RESTORE
Preserve live timer state	Protect active sessions	Create local data copy	Keep pre-restore backup

ENGINEERING CHOICES

A Few Design Decisions I’m Proud Of

Cached page state

Major pages are cached instead of recreated on every navigation change, allowing Pomodoro, Stopwatch and Focus sessions to keep their live state while the user moves around the application.

Explainable productivity score

The score is based on visible rules rather than an arbitrary rating: task completion contributes 50%, Focus time 30%, and Planner completion 20%.

Offline-first storage

Tasks, notes, plans and timing history live in a local SQLite database, while appearance and export preferences are stored locally in JSON.

Responsive desktop UI

The interface uses responsive cards, adaptive layouts, scroll handling and a collapsible sidebar so the application remains usable across different Windows sizes.

SCOPE

What the Project Includes - and What It Does Not

Included

✓ Task CRUD, priorities, categories and date filters

✓ Interactive planner with scheduled activities

✓ Notes with search, categories and full-note popup editing

✓ Focus Mode with navigation protection and partial-session saving

✓ Pomodoro and stopwatch session tracking

✓ Dashboard, productivity score and visual analytics

✓ Categorized history with date filters

✓ Daily, Weekly and Monthly TXT reports

✓ Dark / Light / System themes and responsive UI

✓ SQLite backup / restore and PyInstaller EXE packaging

Intentionally out of scope

– Cloud synchronization

– User accounts or online authentication

– Mobile / web companion application

– Calendar-service integrations

– Recurring task automation

– Push notifications and reminders

– AI productivity recommendations

– Team collaboration or enterprise administration

REFLECTION

What I Learned

LifeOS helped me think beyond individual widgets and functions. The most useful lessons came from connecting persistent data, timer state, navigation, responsive layouts and analytics so that every module feels like part of the same application.

Desktop UI architecture

Reusable pages, components, responsive layouts and navigation.

SQLite integration

CRUD operations, timestamps, filters and report-oriented queries.

Timer state management

Persistent Focus, Pomodoro and Stopwatch behavior across navigation.

Data visualization

Matplotlib/Seaborn charts driven by real application activity.

UX integration

Scrolling, dialogs, theme switching and workflow protection.

Project delivery

Git, README, documentation, PyInstaller packaging and portfolio presentation.

CONCLUSION

Final Thoughts

LifeOS began as a Python mini-project, but it became a practical exercise in desktop application architecture. It combines GUI development, SQLite, timers, file handling, data visualization and local configuration without depending on a web server or cloud service.

For me, the strongest part of the project is the flow between modules: finishing a task changes statistics and reports; focused work contributes to productivity; planner completion changes the daily score; and the same history can be reviewed later. That connected flow is what makes LifeOS feel like a system rather than a set of demos.

EXPLORE

Project Links

GitHub	https://github.com/tushar-011
LinkedIn	https://www.linkedin.com/in/tushar-thakur-8848a7396/
Email	artificial.thakur@gmail.com